

```
In [1]: # Importing libraries

import pandas as pd
import numpy as np
import os
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [2]: path = r'/Users/matthewabrams/Desktop/Stocks Data/'
stocks = pd.read_csv(os.path.join(path, 'Stock Datasets/New Files/6_Stocks_Trans
stocks.drop(columns = ['Unnamed: 0', 'MA_Six_Months', 'MA_One_Year'], inplace = T
stocks
```

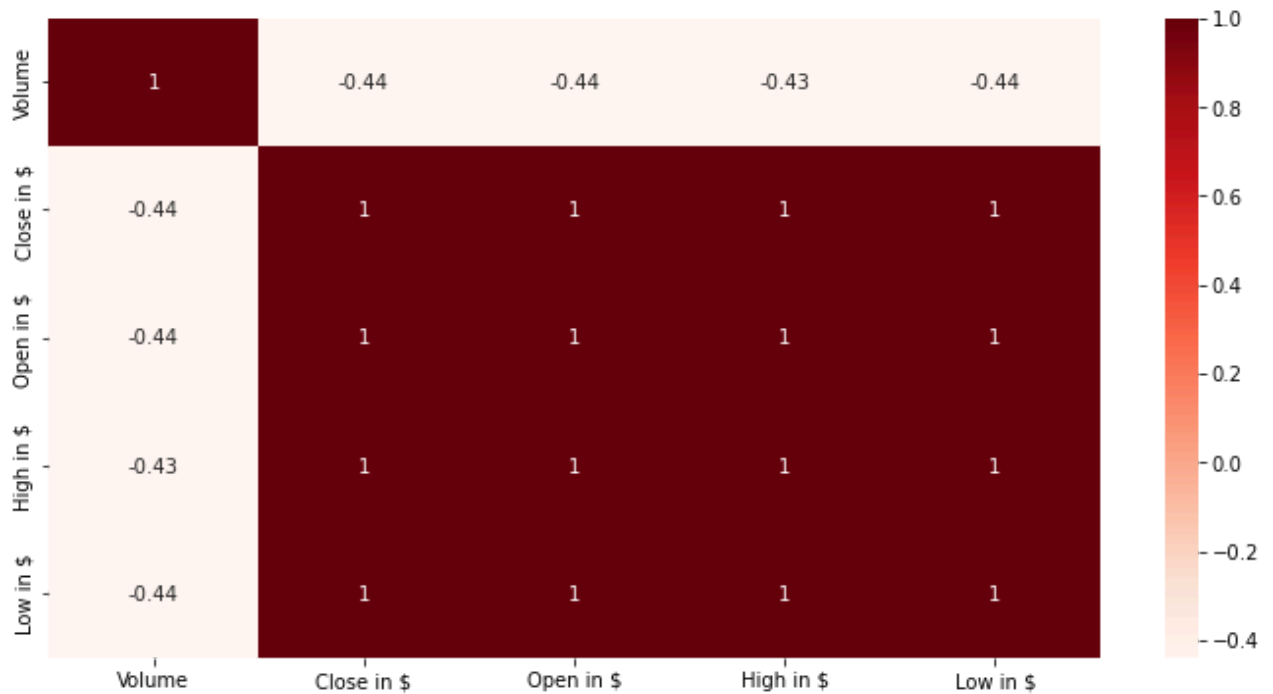
Out[2]:

	Date	Stock_Name	Volume	Close in \$	Open in \$	High in \$	Low in \$
0	2015-01-02	Amazon	55664000	15.4260	15.6290	15.7375	15.3480
1	2015-01-05	Amazon	55484000	15.1095	15.3505	15.4190	15.0425
2	2015-01-06	Amazon	70380000	14.7645	15.1120	15.1500	14.6190
3	2015-01-07	Amazon	52806000	14.9210	14.8750	15.0640	14.7665
4	2015-01-08	Amazon	61768000	15.0230	15.0160	15.1570	14.8055
...	...	...	...	...	...	...	...
15091	2024-12-24	Tesla	59551800	462.2800	435.9000	462.7800	435.1400
15092	2024-12-26	Tesla	76366400	454.1300	465.1600	465.3300	451.0200
15093	2024-12-27	Tesla	82666800	431.6600	449.5200	450.0000	426.5000
15094	2024-12-30	Tesla	64941000	417.4100	419.4000	427.0000	415.7500
15095	2024-12-31	Tesla	76825100	403.8400	423.7900	427.9300	402.5400

15096 rows x 7 columns

```
In [3]: # Heat Map for Visualization Correlations

plt.figure(figsize=(12,6))
sns.heatmap(stocks.corr(),cmap = 'Reds', annot = True)
plt.show()
```



```
In [4]: # Splitting the dataset 80% training and 20% testing
X = stocks[['Volume', 'Open in $', 'High in $', 'Low in $']].values
y = stocks['Close in $'].values
```

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

```
In [6]: # Creating regression model
regressor = LinearRegression()

# Fit linear regression model
model = regressor.fit(X_train, y_train)

# Use model to make prediction
y_pred = regressor.predict(X_test)
```

```
In [7]: # With the test predictions complete, next step is to compare them (table of Actual vs Predicted)
# With the actual output values for X_test by organizing them in a dataframe form

predicted = regressor.predict(X_test)
dframe = pd.DataFrame(y_test, predicted)
dfr = pd.DataFrame({'Actual_Price': y_test, 'Predicted_Price': predicted})
print(dfr)
```

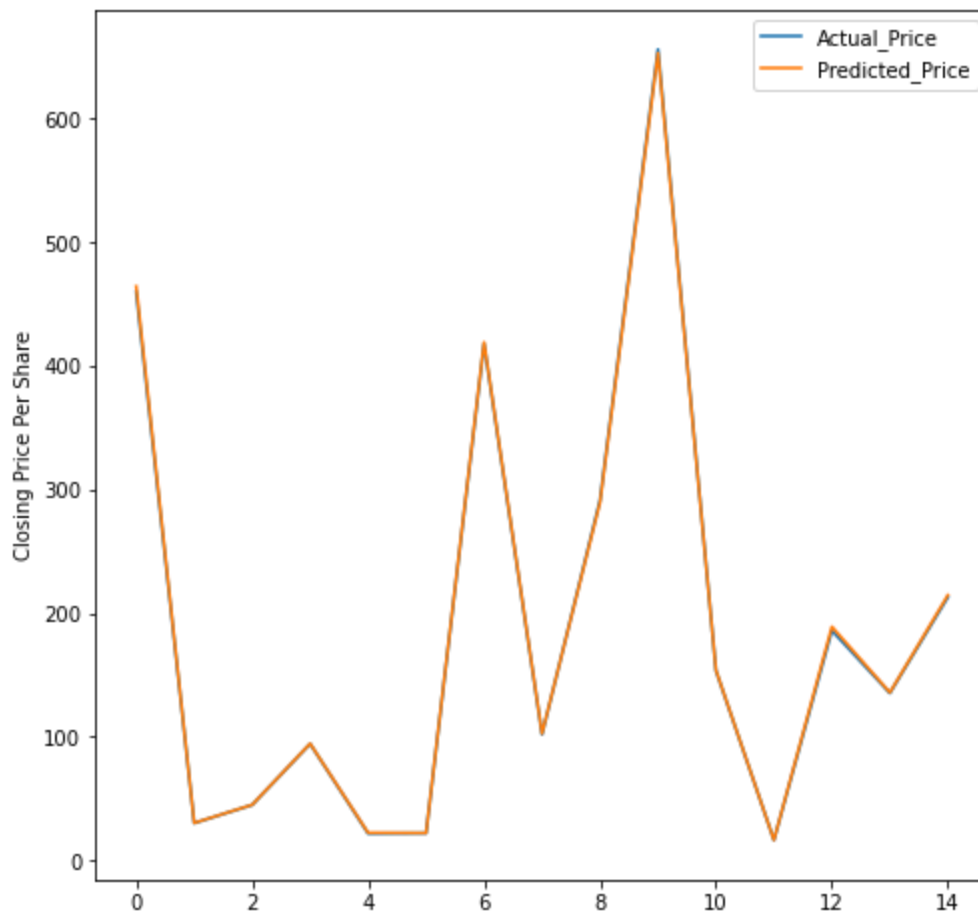
	Actual_Price	Predicted_Price
0	460.1200	464.537342
1	30.2650	30.400635
2	44.9600	45.104638
3	94.3730	94.623203
4	21.8473	22.463867
...	...	...
3015	203.9000	202.721840
3016	505.5200	503.246138

3017	271.8700	273.105539
3018	168.8400	168.732314
3019	62.1400	62.245401

[3020 rows x 2 columns]

```
In [8]: plt.figure(figsize = (8,8))
plt.ylabel('Closing Price Per Share')
plt.plot(dfr.head(15))
plt.legend(['Actual_Price', 'Predicted_Price'])
```

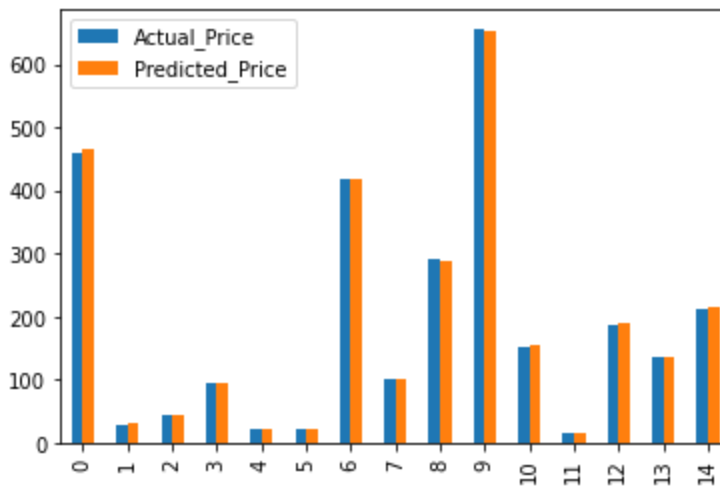
```
Out[8]: <matplotlib.legend.Legend at 0x7fbed8cc1eb0>
```



```
In [9]: # Graph first 15 values

graph = dfr.head(15)
graph.plot(kind = 'bar')
```

```
Out[9]: <AxesSubplot:>
```



```
In [10]: # The predicted prices are extremely close to the actual prices
```

```
In [11]: # Extreme Value Checks
```

```
In [12]: stocks['Close in $'].mean()
```

```
Out[12]: 176.53533192898794
```

```
In [13]: stocks['Close in $'].std()
```

```
Out[13]: 146.96241625340735
```

```
In [14]: # Determining upper limit for outliers that are 2 standard deviations of the mean

upper_limit_1 = stocks['Close in $'].mean() + 2*(stocks['Close in $'].std())
upper_limit_1
```

```
Out[14]: 470.46016443580265
```

```
In [15]: # Determining lower limit for outliers that are 2 standard deviations of the mean

lower_limit_1 = stocks['Close in $'].mean() - 2*(stocks['Close in $'].std())
lower_limit_1
```

```
Out[15]: -117.38950057782677
```

```
In [16]: stocks['Close in $'].max()
```

```
Out[16]: 936.56
```

```
In [17]: # Determining the percentage of values under 'Close/Last in $' column that are not outliers
# Dividing the number of values between 0 and upper limit and the total number of values
```

```
len(stocks[stocks['Close in $']<upper_limit_1])/len(stocks)
```

Out[17]: 0.9428987811340752

```
In [18]: # About 6% of values under Close/Last in $ column are outliers
```

```
In [19]: stocks['Volume'].mean()
```

Out[19]: 61034213.93746688

```
In [20]: stocks['Volume'].std()
```

Out[20]: 62560077.65000739

```
In [21]: # Determining upper limit for outliers that are 2 standard deviations of the mean  
  
upper_limit_2 = stocks['Volume'].mean() + 2*(stocks['Volume'].std())  
upper_limit_2
```

Out[21]: 186154369.23748165

```
In [22]: # Determining upper limit for outliers that are 2 standard deviations of the mean  
  
lower_limit_2 = stocks['Volume'].mean() - 2*(stocks['Volume'].std())  
lower_limit_2
```

Out[22]: -64085941.3625479

```
In [23]: stocks['Volume'].max()
```

Out[23]: 914082000

```
In [24]: # Determining the percentage of values under Volume column that are not outliers  
# Dividing the number of values between 0 and upper limit and the total number of values  
  
len(stocks[stocks['Volume']<upper_limit_2])/len(stocks)
```

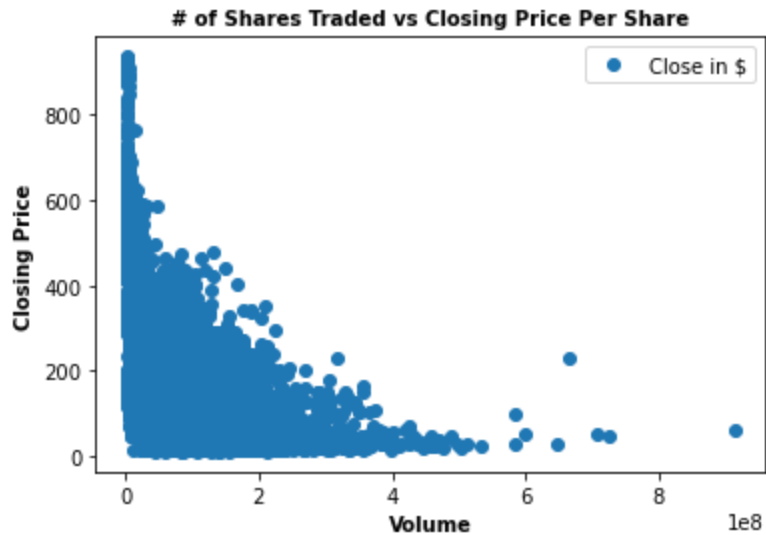
Out[24]: 0.9548224695283519

```
In [25]: # About 5% of values under Volume in $ column are outliers
```

```
In [26]: # Original Hypothesis: If the closing price is low, more shares are traded
```

```
In [27]: # Create a scatterplot using matplotlib for another look at how the chosen variables
```

```
stocks.plot(x = 'Volume' , y = 'Close in $', style='o') # The style option creates a scatter plot
plt.title('# of Shares Traded vs Closing Price Per Share', fontsize = 10, fontweight = 'bold')
plt.xlabel('Volume', fontsize = 10, fontweight = 'bold')
plt.ylabel('Closing Price', fontsize = 10, fontweight = 'bold')
plt.show()
```



In [28]: *# Reshape the variables into NumPy arrays and put them into separate objects.*

```
X1 = stocks['Volume'].values.reshape(-1,1)
y1 = stocks['Close in $'].values.reshape(-1,1)
```

In [29]: *# Split data into a training set and a test set.*

```
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2, random_state=42)
```

In [30]: *# Create a regression object.*

```
regression = LinearRegression() # This is the regression object, which will be used to fit the data
# Fit the regression object onto the training set.
regression.fit(X1_train, y1_train)
```

Out[30]: LinearRegression()

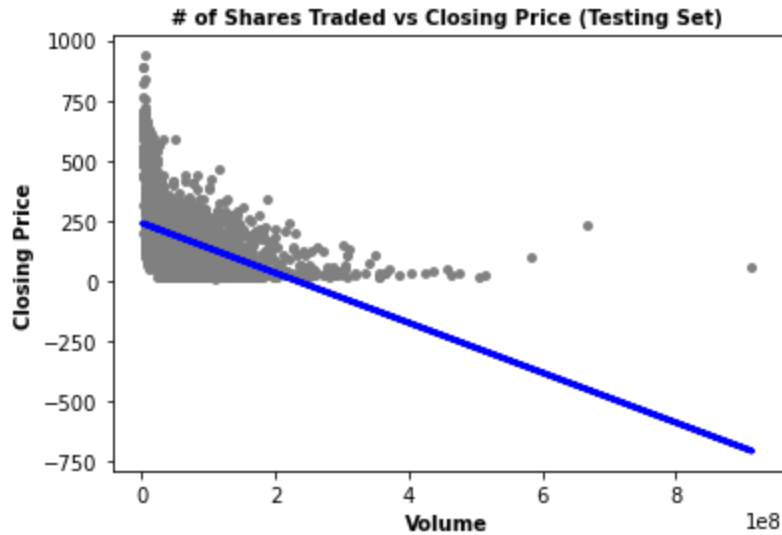
In [31]: *# Predict the values of y using X.*

```
y1_predicted = regression.predict(X1_test)
```

In [32]: *# Create a plot that shows the regression line from the model on the test set.*

```
plot_test = plt
plot_test.scatter(X1_test, y1_test, color= 'gray', s = 15)
plot_test.plot(X1_test, y1_predicted, color= 'blue', linewidth = 3)
plot_test.title('# of Shares Traded vs Closing Price (Testing Set)', fontsize = 10, fontweight = 'bold')
plot_test.xlabel('Volume', fontsize = 10, fontweight = 'bold')
```

```
x_ticks = np.arange(0, 300, 50)
plot_test.ylabel('Closing Price', fontsize = 10, fontweight = 'bold')
plot_test.show()
```



```
In [33]: # Create objects that contain the model summary statistics.

rmse = mean_squared_error(y1_test, y1_predicted).round() # This is the mean squared error
r2 = r2_score(y1_test, y1_predicted).round(2) # This is the R2 score.
```

```
In [34]: # Print the model summary statistics. This is where you evaluate the performance

print('Slope:', regression.coef_.round(3))
print('Mean squared error: ', rmse)
print('R2 score: ', r2)
```

```
Slope: [[-0.]]
Mean squared error: 18144.0
R2 score: 0.18
```

```
In [35]: # Since this linear regression model is about 18% effective for my testing set,
# This could be due to data bias or not enough data points to produce significant correlation
# I'll need a model with more dimensionality to be able to predict what would effect the closing price
```

```
In [36]: # Create a dataframe comparing the actual and predicted values of y.

data = pd.DataFrame({'Actual': y1_test.flatten(), 'Predicted': y1_predicted.flatten()})
data.head(30)
```

```
Out[36]:
```

	Actual	Predicted
0	460.1200	217.548874
1	30.2650	102.201955
2	44.9600	179.718664
3	94.3730	172.541921
4	21.8473	-54.236228

	Actual	Predicted
5	22.1400	34.205057
6	418.0100	213.830176
7	102.0500	161.757003
8	291.5600	231.136683
9	656.3200	236.450708
10	153.5900	220.521282
11	16.5653	197.104797
12	186.0600	195.187958
13	135.4500	150.144924
14	213.0200	208.669494
15	19.1830	171.677228
16	15.9920	168.115813
17	170.5500	187.587577
18	214.5800	225.643602
19	314.6600	236.734791
20	337.9100	216.776767
21	495.0600	222.967305
22	133.5800	127.509359
23	57.3200	136.433425
24	172.0900	224.817167
25	240.0200	229.708177
26	424.7700	131.109654
27	17.4713	183.468777
28	18.6685	184.272299
29	41.7885	177.054083

In [37]: *# The predicted values are extremely far from the actual values*

In [38]: *# Compare how the regression fits the training set*
`y1_predicted_train = regression.predict(X1_train)` *# This is predicting X_train!*

In [39]: `rmse = mean_squared_error(y1_train, y1_predicted_train).round()`
`r2 = r2_score(y1_train, y1_predicted_train).round(2)`

In [40]: `print('Slope:', regression.coef_.round(3))`
`print('Mean squared error: ', rmse)`


```
print('R2 score: ', r2)
```

Slope: $[-0.]$

Mean squared error: 17308.0

R2 score: 0.19

In [41]:

```
# Visualizing the training set results.
```

```
plot_test = plt
plot_test.scatter(X1_train, y1_train, color='red', s = 15)
plot_test.plot(X1_train, y1_predicted_train, color='blue', linewidth = 3)
plot_test.title('# of Shares Traded vs Closing Price (Training Set)', fontsize =
plot_test.xlabel('Volume', fontsize = 10, fontweight = 'bold')
x_ticks = np.arange(0, 300, 50)
plot_test.ylabel('Closing Price', fontsize = 10, fontweight = 'bold')
plot_test.show()
```



In [42]:

```
# This linear regression model is about 19% effective for my training set, about
```